

Analysis of Type-Driven approach to systems programming: Implementation of OpenGL library for Rust

Author: Mikołaj Depta

Supervisor: dr. Andrzej Łukaszewski

Date: 03.09.2024

Github: <https://github.com/mikiSpoko200/gpu-bulwark>

Agenda

- Introduction (3min)
 - Type-Driven Design
 - Systems programming
 - Why Rust
 - Why OpenGL
 - Project's goals
 - Key features
- Key features and examples (3min) *
- Conclusions (1min)
 - What was achieved
 - Current limitations
 - Future plans

Introduction

Type-Driven Design

- **Expressing Contracts with Types:** Using types to define precise contracts for data structures, functions and modules
- **Data Invariants:** Leveraging types to enforce constraints at compile-time, ensuring that certain properties hold throughout the program (e.g., positive numbers, non-empty lists)
- **Type-Safe APIs:** Representing data states explicitly with types along with specifying state transitions, making invalid application states unrepresentable
- **Static Property Enforcement:** Using types to express static properties (e.g., immutability, range restrictions) guaranteeing correctness at run-time

Systems programming

- **Robust solutions:** Creating programs resistant to misuse
- **System resource management:** Manual handling of limited system resources in a safe and efficient manner
- **Concurrency and Parallelism:** Working with need for explicit synchronization
- **Interoperability with External Components:** Facilitating communication with other software, hardware, and external systems - integration with different binary protocols
- **Performance Optimization:** Ensuring efficient, low-latency, and high-throughput execution

Why Rust?

Rust is a modern, statically typed, systems programming language.

At the same time it's a relatively new language (1.0 in 2015), the goal of this study was to showcase that it can be successfully applied to graphics programming.

Notable features:

- State of the art type system
- Natively compiled, with very good support for C interoperability
- Precise semantics of resource management
- **borrow checker** allows to cleanly express how different pieces of program interact with each other
- Large ecosystem of open-source packages

Why OpenGL?

OpenGL is a very mature, cross-platform, well understood and widely supported graphics specification. It's arguably the best API for beginners in hardware-accelerated graphics programming.

However, it's also an old C API, with global state mutation at its core* and very limited debugging facilities. Certain aspects of the programming interface make using OpenGL difficult despite its simple execution model.

We felt like Rust's type system could mitigate some of these difficulties, providing an alternative for beginners to get into computer graphics without spending weeks on getting used to writing OpenGL.

Project's Goals and key features

There were two main goals for this study:

- Explore type-driven design in Rust
- Implement a minimalistic OpenGL wrapper

Explore type-driven design in Rust

- Determine benefits and downsides of such an approach to systems programming
- Ascertain feasibility of developing software that way
- Identify common patterns for type-driven design

Implement a minimalistic OpenGL wrapper

- Remain as close to original specification as possible
- Implement only the most essential functionality
- Prevent as many ill-formed programs at compile-time as possible
- Finally, optimize UX aspects

Computer Graphics primer (in 30 seconds)

API specifications for hardware accelerated graphics

Graphics Processing Units (GPUs) are controlled via specification like OpenGL, Vulkan or DirectX.

GPU vendors implement specifications and software developers program against said specifications.

This makes code reusable across different devices.

Programmable graphics pipeline

Hardware rendering is quite a complicated process. It requires many sequential stages (hence pipeline); some of which are **programmable**.

Programs that specify the behavior of a programmable stage are called **shaders**.

Shaders are written in specialized languages like **GLSL** or **HLSL**, which don't directly control GPU's resources - they only declare what they expect to be given. Thus, a need for **interoperability** arises since the programmer must manually guarantee that correct data is present at render-time.

Shaders

Vertex shader

```
1 #version 420 core
2
3 layout(location = 0) in vec3 in_color;
4 layout(location = 1) in vec3 in_position;
5
6 layout(location = 0) out vec4 color;
7
8 void main() {
9     gl_Position = vec4(in_position, 1.0);
10    color = vec4(in_color, 1.0);
11 }
```

Fragment shader

```
1 #version 420 core
2
3
4 layout(location = 0) in vec4 color;
5 layout(location = 0) out vec4 out_color;
6
7
8 void main() {
9     out_color = color;
10 }
```

Key features and examples

GLSL DSL macros

In order to facilitate seamless interoperability macros providing GLSL domain specific languages were developed.

```
1 type VertexShaderInputs = glsl::Gls1! {  
2     layout(location = 0) in vec3 in_position;  
3     layout(location = 1) in vec3 in_normal;  
4     layout(location = 2) in vec3 in_tex_coordinates;  
5 };  
6  
7 fn update_uniforms(new_matrix: [[f32; 4]; 4]) {  
8     // ...  
9     let glsl::vars! [.mvp_matrix, dt] = gb::glsl! {  
10         layout(location = 0) in mat4;  
11         layout(location = 1) in float;  
12     };  
13     program.uniform(&mvp_matrix, &new_matrix);  
14     // ...  
15 }
```

```
1 #version 420 core  
2  
3 layout(location = 0) in vec3 in_position;  
4 layout(location = 1) in vec3 in_normal;  
5 layout(location = 2) in vec3 in_tex_coordinates;  
6  
7 // outputs ..  
8  
9 layout(location = 0) uniform mat4.mvp_matrix;  
10 layout(location = 1) uniform float dt;  
11  
12 void main() {  
13     // ...  
14 }
```


Compile-time guarantees

```
1 #version 420 core
2
3 layout(location = 0) in vec3 in_color;
4 layout(location = 1) in vec3 in_position;
5
6 layout(location = 0) out vec4 color;
7
8 void main() {
9     gl_Position = vec4(in_position, 1.0);
10    color = vec4(in_color, 1.0);
11 }
```

1. Data compatibility with shader declarations

Keeping track of shape of a N-dimensional arrays is a tedious and error prone process. Momentary lack of focus may cause invalid data to be provided to a routine. This will usually cause a run-time error, but since OpenGL exposes a C API and is provided as a binary for linking by the OS, it cannot use type information to bound check passed data.

1. Data compatibility with shader declarations

```
1 Buffer<int> colorBuffer = Buffer<float>::Array();
2 const std::vector<int> colors = {
3     1, 0, 0, 1,
4     0, 1, 0, 1,
5     0, 0, 1, 1,
6 };
7
8 colorBuffer.Data(colors, GL_STATIC_DRAW);
9 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT);
```

```
1 let glsl::vars! [ vin_color, vin_position ] = gb::glsl!
2     layout(location = 0) in vec3;
3     layout(location = 1) in vec3;
4 };
5 let mut colors = Buffer::create();
6
7 colors.data::<(Dynamic, Draw)>(&[
8     [1, 0, 0, 1],
9     [0, 1, 0, 1],
10    [0, 0, 1, 1i32]
11 ]);
12
13 let vao: VertexArray<Attributes> = VertexArray::create()
14     .vertex_attrib_pointer(&vin_color, colors);
```

1. Data compatibility with shader declarations

```
error[E0308]: mismatched types
--> src\listing.rs:101:44
101 |         let vao: VertexArray<Attributes> = VertexArray::create()
    |                                ^
    |
    | expected due to this
102 |         .vertex_attrib_pointer(&vin_color, colors)
103 |         .vertex_attrib_pointer(&vin_position, positions)
    |         ^ expected `f32`, found `i32`
    = note: expected struct `VertexArray<(((), Attribute<[f32; 3], _>), Attribute<_, _>)>`
           found struct `VertexArray<(((), Attribute<[i32; 4], _>), Attribute<_, _>)>`
```

2. Valid resource locations

Another bug strong typing can detect, is using attribute index which does not match with vertex shader input variables.

```
1 #version 420 core
2
3 layout(location = 0) in vec3 in_color;
4 layout(location = 1) in vec3 in_position;
5
6 layout(location = 0) out vec4 color;
7
8 void main() {
9     gl_Position = vec4(in_position, 1.0);
10    color = vec4(in_color, 1.0);
11 }
```

2. Valid resource locations

```
1 colorBuffer.Data(colors, GL_STATIC_DRAW);
2 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT);
3
4 positionBuffer.Data(positions, GL_STATIC_DRAW);
5 vao.VertexAttribPointer(0, positionBuffer, 3, GL_FLOAT);
```

```
1 let glsl::vars! [ vin_color, vin_position ] = gb::glsl! {
2     layout(location = 0) in vec3;
3     layout(location = 1) in vec3;
4 };
5
6 let vao: VertexArray<Attributes> = VertexArray::create(
7     .vertex_attrib_pointer(&vin_color, colors)
8     .vertex_attrib_pointer(&vin_color, positions)
9     ;
```

2. Valid resource locations

```
error[E0308]: mismatched types
--> src\listing.rs:105:36

8 |         .vertex_attrib_pointer(&vin_color, positions)
  |         ^^^^^^^^^^^^^^^^^^^^^ expected `1`, found `0`
  |         arguments to this method are incorrect
  |
  = note: expected reference `&gpu_bulwark::glsl::Variable<_, gpu_bulwark::glsl::layout::Location<1>, _>`
found reference `&gpu_bulwark::glsl::Variable<_, gpu_bulwark::glsl::layout::Location<0>, GVec<f32, 3>>`
```

3. Usage of only compiled shaders

Type parameters along with Rust's method dispatch mechanism can be used to implement a type-level state machine.

```
mod ts {  
    #[hi::mark(InstanceState, Compilation)]  
    pub enum Uncompiled { }  
  
    #[hi::mark(InstanceState, Compilation)]  
    pub struct Compiled;  
}
```


3. Usage of only compiled shaders

```
1 impl Shader<ts::Uncompiled, ...> {
2     /// Create a shader object, in uncompiled state.
3     pub fn create() -> Self { ... }
4
5     /// Add source code.
6     pub fn source(&mut self, sources: &[&str]) -> &Self { ... }
7
8     /// Try compiling.
9     pub fn compile(self) -> Result<Shader<ts::Compiled, ...>, CompilationError> { ... }
10 }
11
12 impl Shader<ts::Compiled, ...> {
13     pub fn into_main(self) -> Main<...> { ... }
14 }
```

3. Usage of only compiled shaders

```
1 impl Shader<ts::Uncompiled, ...> {
2     /// Create a shader object, in uncompiled state.
3     pub fn create() -> Self { ... }
4
5     /// Add source code.
6     pub fn source(&mut self, sources: &[&str]) -> &Self { ... }
7
8     /// Try compiling.
9     pub fn compile(self) -> Result<Shader<ts::Compiled, ...>, CompilationError> { ... }
10 }
11
12 impl Shader<ts::Compiled, ...> {
13     pub fn into_main(self) -> Main<...> { ... }
14 }
```

3. Usage of only compiled shaders

```
1 pub fn try_using_uncompiled() {  
2     let mut shader = shader::create::<shader::target::Vertex>();  
3     shader.into_main();  
4 }
```

```
error[E0599]: no method named `into_main` found for struct `Shader<Uncompiled, Vertex, ()>` in the current scope  
--> src\example_wrong_buffers.rs:107:12  
3 |     shader.into_main();  
  |           ^^^^^^^^^ method not found in `Shader<Uncompiled, Vertex, ()>`  
= note: the method was found for  
       - `Shader<Compiled, T, Decl>`
```

4. Correct pipeline configuration

```
1 let vs = uncompiled_vs
2   .uniform(&view_matrix_location)
3   .uniform(&scale_location)
4   .compile()?
5   .into_main()
6   .inputs(&vs_inputs)
7   .outputs(&vs_outputs);
8 let fs = uncompiled_fs
9   .compile()?
10  .into_main()
11  .inputs(&fs_inputs)
12  .output(&fs_output);
13 let common = common.compile()?.into_shared();
```

```
1 let program = Program::builder()
2   .uniforms(|definitions| definitions
3     .define(&view_matrix_location, &matrix)
4     .define(&scale_location, &scale)
5   )
6   .no_resources()
7   .vertex_main(&vs)
8   .uniforms(|matcher| matcher
9     .bind(&scale_location)
10    .bind(&view_matrix_location)
11  )
12  .vertex_shared(&common)
13  .fragment_main(&fs)
14  .build()?;
```

Recursive types

Rust associates types with their functionality using `impl` blocks. These can be generic and apply to all types that satisfy given constraints.

```
1 pub MyTrait { }  
2  
3 impl<T> MyTrait for T where T: Debug { }
```

Recursive types

```
1 trait HList {  
2     const SIZE: usize;  
3 }  
4  
5 impl HList for () {  
6     const SIZE: usize = 0;  
7 }  
8  
9 impl<Head, Tail> HList for (Head, Tail)  
10 where  
11     Tail: HList  
12 {  
13     const SIZE: usize = Tail::SIZE + 1;  
14 }
```

Recursive types - DSL macro expansion

```
1 gb::gls1! {  
2     layout(location = 0) in mat4;  
3     layout(location = 4) in vec4;  
4 };
```

Recursive types - DSL macro expansion

```
1 gb::constraint::ValidExt::validated((
2   (gb::constraint::ValidExt::validated((
3     (),
4     gb::glsl::variable::Variable::<
5       gb::glsl::variable::storage::In,
6       gb::glsl::variable::layout::Location<0>,
7       gb::glsl::Mat4,
8     >::default(),
9   )),
10  gb::glsl::variable::Variable::<
11    gb::glsl::variable::storage::In,
12    gb::glsl::variable::layout::Location<4>,
13    gb::glsl::Vec4,
14  >::default(),
15 ))
```


Const parameterized types - DSL macro expansion

```
1 gb::constraint::ValidExt::validated((
2   (gb::constraint::ValidExt::validated((
3     (),
4     gb::glsl::variable::Variable::<
5       gb::glsl::variable::storage::In,
6       gb::glsl::variable::layout::Location<0>,
7       gb::glsl::Mat4,
8     >::default(),
9   )),
10  gb::glsl::variable::Variable::<
11    gb::glsl::variable::storage::In,
12    gb::glsl::variable::layout::Location<4>,
13    gb::glsl::Vec4,
14  >::default(),
15 ))
```

Conclusions

OpenGL Wrapper

- Library implements the most essential parts of the OpenGL spec
- Library statically detects misuse of the OpenGL API
- Successful compilation guarantees correct configuration of the graphics pipeline
- Resulting code closely resembles analogous C / C++ code
- Type errors help to identify the cause of the problem

Type-driven design

- The biggest benefit of extensive use of the type system is the static verification of the correctness of the analyzed programs
- Type-driven design forces greater attention during the initial phase of software creation
- The logical principles underpinning the type system allow for a very intuitive description of the program's structure, algorithm invariants, as well as the interactions between different components
- Enforcing greater attention during system architecture design brings significant benefits in later stages of development

Rust for OpenGL

- Interfacing with OS platform will be inherently unsafe
- Rust is a viable technology for programming computer graphics
- With initial overhead of providing safe facades around raw OS APIs, development becomes simpler than in C or C++
- Rust's type system can be used to express data formats to external APIs

Innovative approach to computer graphics

- Type-Safe API Design
- A new way to learn Graphics Programming
- Precision in Interoperability
- Novelty in Combining Disciplines

Cross disciplinary

- Programming unsafe Rust requires a great deal of attention, as Rust heavily exploits assumption of no Undefined Behavior
- Programming language theory - in-depth understating of Rust's type system and its limitations, studying numerous RFCs
- Software design - a great deal of attention was put towards designing API up to common engineering standards

Current limitations

- No `impl` specialization
- No negative trait bounds
- No `const` generics in `const` expressions
- No `const` fns in traits

Future plans

Further improvements, community feedback

Publishing to `crates.io`

Source code for this project is already publicly available on github. Next step in proliferating type driven design is publishing this crate to `crates.io`, once more features are added.

One module of this library called [himark](#) was published during development, and as of today has almost 1200 downloads.

Integration with `rust-gpu`

```
use glam::{Vec3, Vec4, vec2, vec3};

#[spirv(fragment)]
pub fn main(
    #[spirv(frag_coord)] in_frag_coord: &Vec4,
    #[spirv(push_constant)] constants: &ShaderConstants,
    output: &mut Vec4,
) {
    let frag_coord = vec2(in_frag_coord.x, in_frag_coord.y);
    let mut uv = (frag_coord - 0.5 * vec2(constants.width as f32, constants.height as f32))
        / constants.height as f32;
    uv.y = -uv.y;

    let eye_pos = vec3(0.0, 0.0997, 0.2);
    let sun_pos = vec3(0.0, 75.0, -1000.0);
    let dir = get_ray_dir(uv, eye_pos, sun_pos);
```

Thank you for your attention

Analysis of Type-Driven approach to systems programming: Implementation of OpenGL library for Rust

Author: Mikołaj Depta

Supervisor: dr. Andrzej Łukaszewski

Date: 03.09.2024

Github: <https://github.com/mikiSpoko200/gpu-bulwark>